Введение в ROS, базовые концепции, первый запуск

На основе лекции О. Шипитько и по материалам семинара Ю. Мурхиж, Л. Абдразаков, И.Белкин. “Особенности реализации алгоритмов CV и SLAM в ROS2”

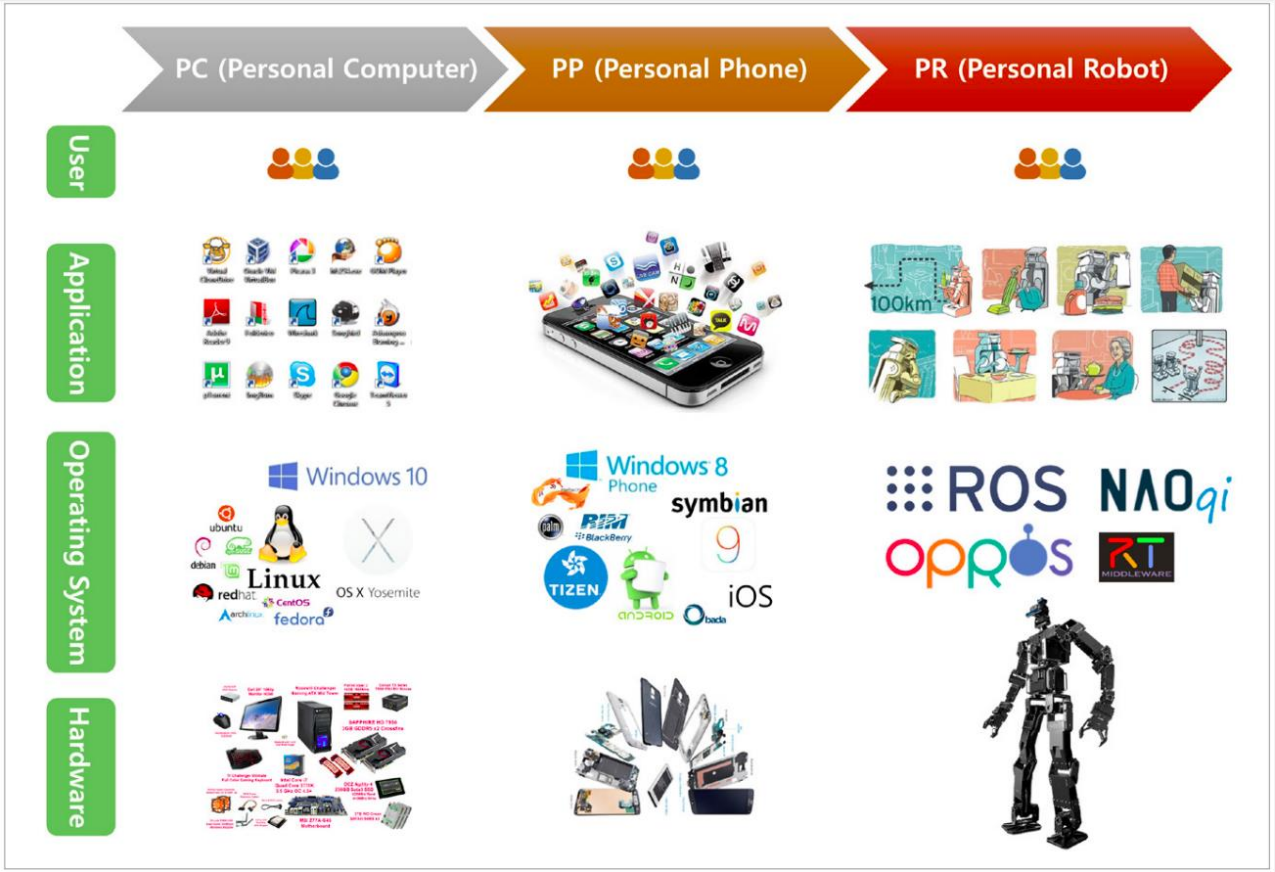
ИППИ РАН

MIPT
MOSCOW INSTITUTE OF PHYSICS AND TECHNOLOGY

СОДЕРЖАНИЕ ЛЕКЦИИ

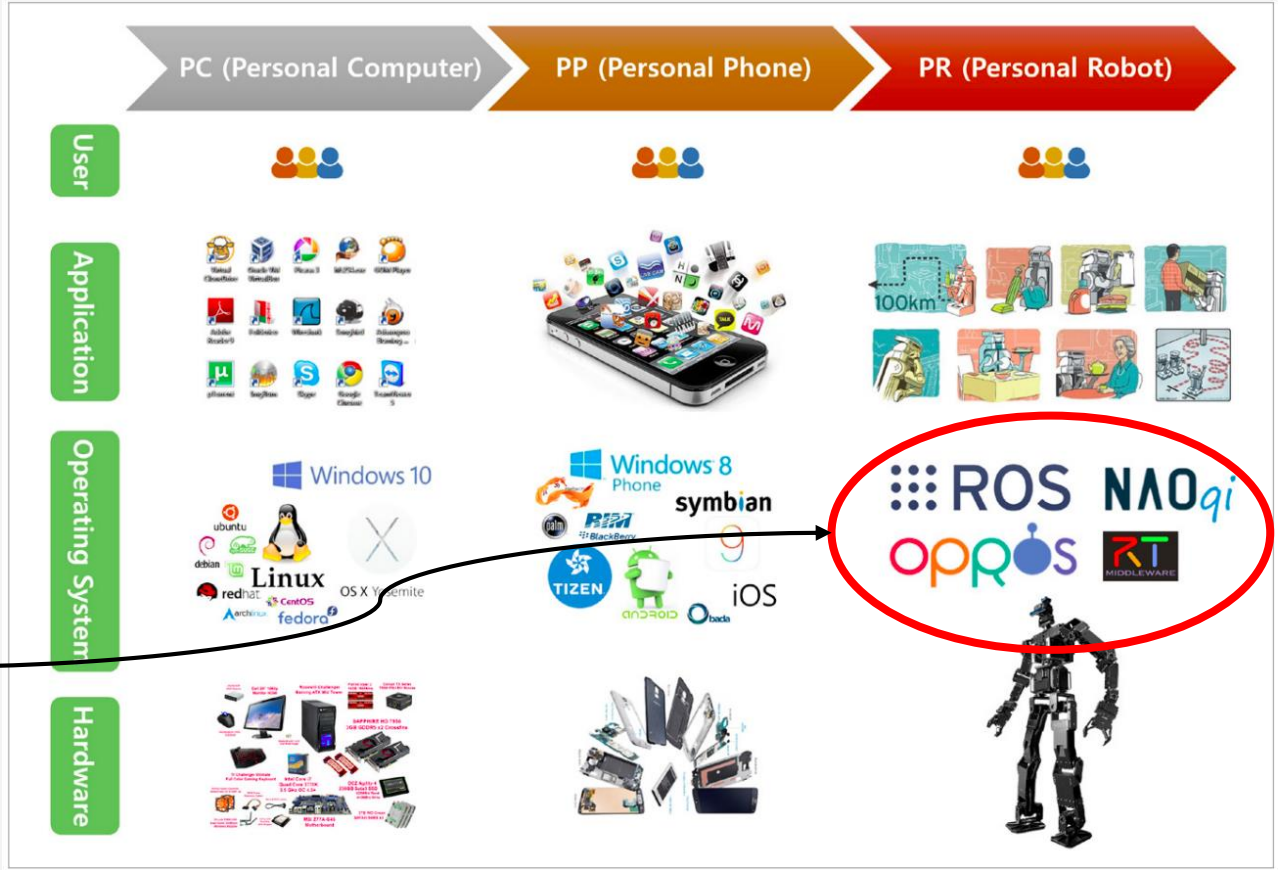
1. Зачем роботам операционная система?
2. Что такое ROS?
3. Место ROS в программировании роботов
4. История создания и существующие дистрибутивы ROS
5. Установка ROS
    a. ROS из контейнера
6. Первый запуск ROS
    a. source...
    b. roscore
        i. rosmaster
        ii. Parameter Server
        iii. rosout
7. Демонстрация запуска готовых модулей на примере модуля turtlesim

ЗАЧЕМ РОБОТАМ ОС?



Источник: ROS Robot Programming. YoonSeok Pyo, HanCheol Cho, RyuWoon Jung, TaeHoon Lim

ЗАЧЕМ РОБОТАМ ОС?

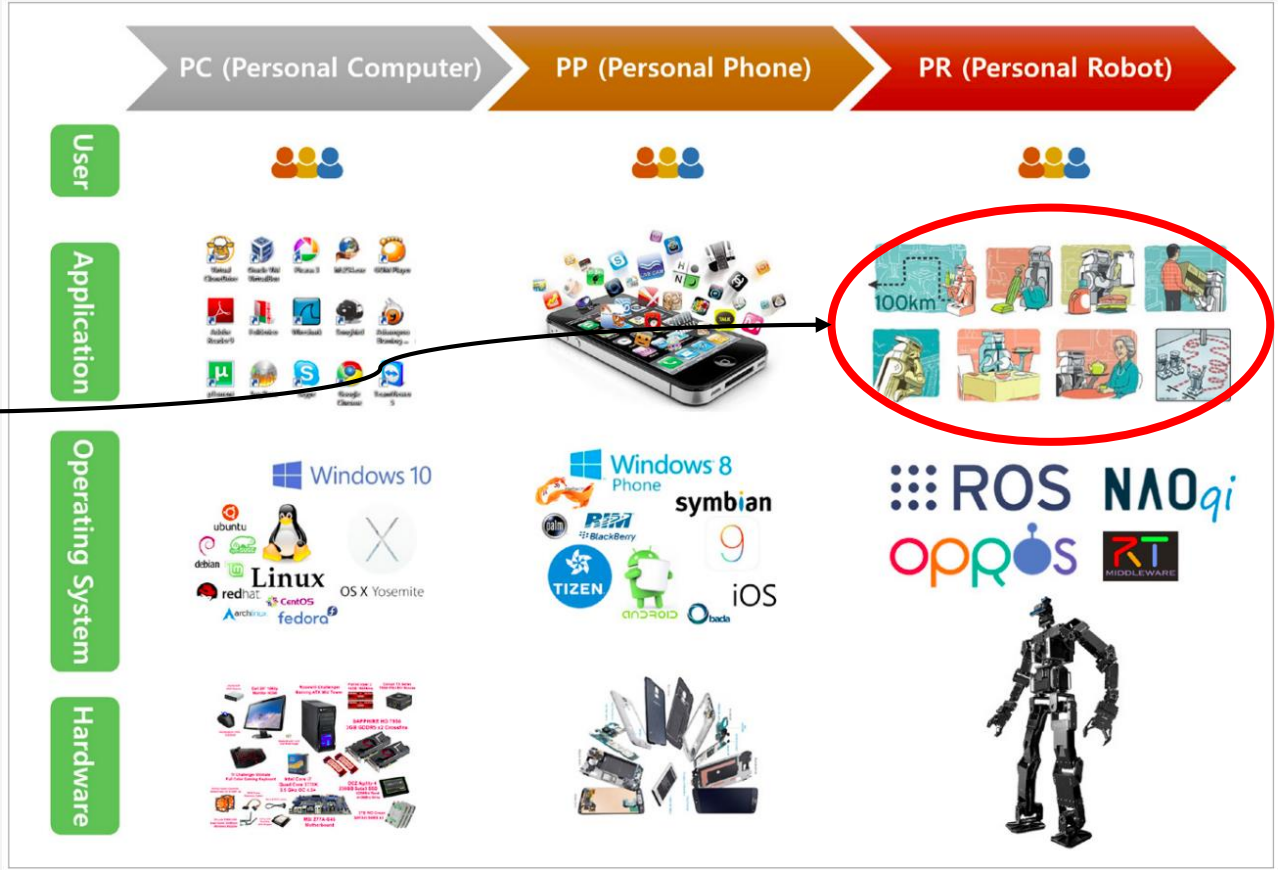


ОС предоставляет абстракцию от “железа”, а также унифицирует процессы передачи данных между процессами (подсистемами)

Источник: ROS Robot Programming. YoonSeok Pyo, HanCheol Cho, RyuWoon Jung, TaeHoon Lim

ЗАЧЕМ РОБОТАМ ОС?

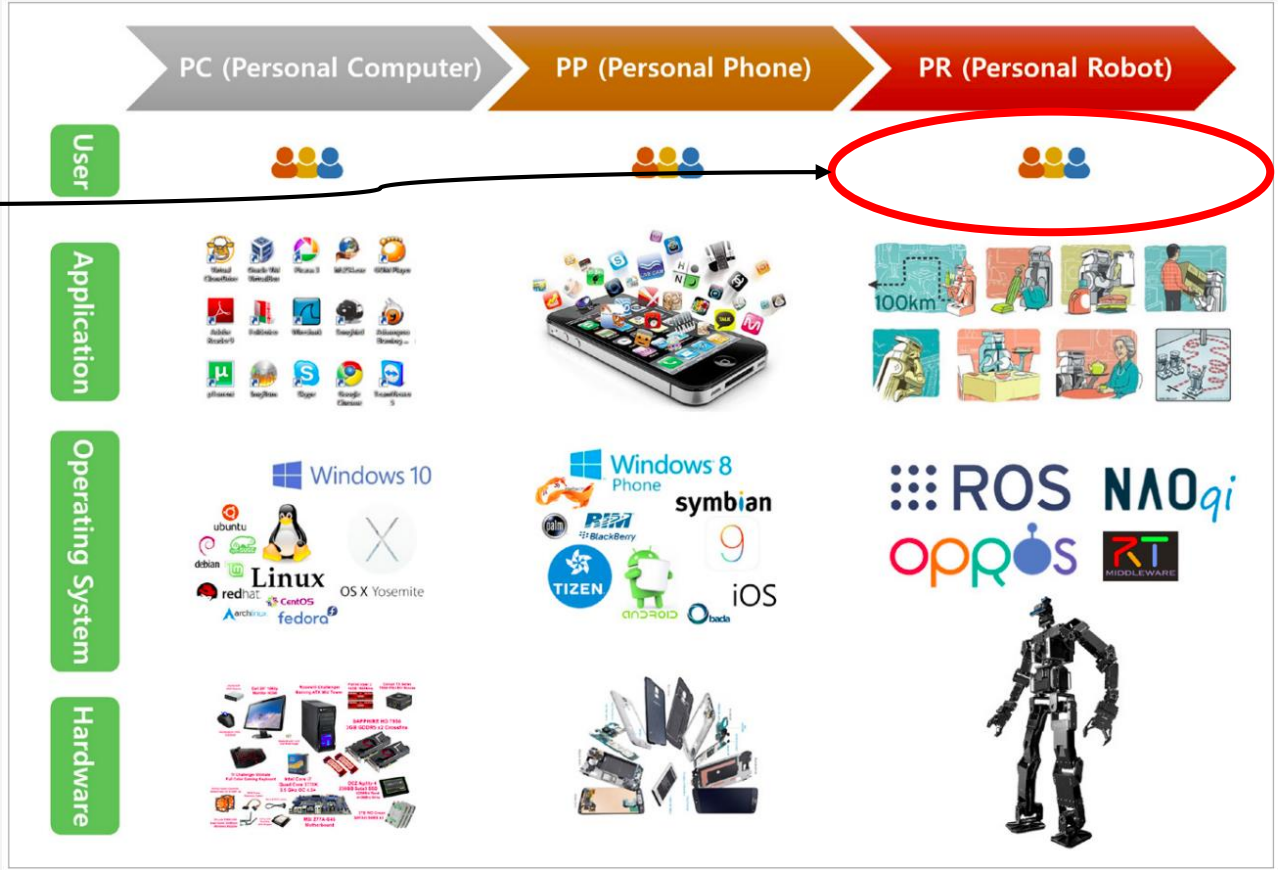
Разработчики приложений могут сосредоточиться на написании ПО



Источник: ROS Robot Programming. YoonSeok Pyo, HanCheol Cho, RyuWoon Jung, TaeHoon Lim

ЗАЧЕМ РОБОТАМ ОС?

Пользователи получают большое разнообразие приложений, работающей на одном физическом устройстве



Источник: ROS Robot Programming. YoonSeok Pyo, HanCheol Cho, RyuWoon Jung, TaeHoon Lim

ПЛЮСЫ ОС В РОБОТОТЕХНИКЕ

- ❏ Абстракция от “железа”
- ❏ Переиспользование программ
- ❏ Модулярность за счет унифицированной парадигмы обмена данными между подпрограммами
- ❏ Инструменты разработки и отладки
- ❏ Сообщество

СУЩЕСТВУЮЩИЕ ОС/ФРЕЙМВОРКИ ДЛЯ РОБОТОВ

- **MSRDS**, Microsoft Robotics Developer Studio
- **ERSP**, Evolution Robotics Software Platform, Evolution Robotics
- **ROS**, Robot Operating System, Open Robotics
- **OpenRTM**, National Institute of Adv. Industrial Science and Technology (AIST)
- **NAOqi OS**, SoftBank and Aldebaran
- ...











Источник: ROS Robot Programming. YoonSeok Pyo, HanCheol Cho, RyuWoon Jung, TaeHoon Lim

ROBOT OPERATING SYSTEM

Robot Operating System, ROS — свободно распространяемая мета-операционная система для роботов. ROS обеспечивает стандартные службы операционной системы:

- ❏ аппаратную абстракцию
- ❏ низкоуровневое управление устройствами
- ❏ готовые реализации часто используемых функций
- ❏ передачу информации между процессами
- ❏ управление пакетами



ROBOT OPERATING SYSTEM. ЭКОСИСТЕМА



- More than 5,000 packages
- 2,818 Official Packages (Indigo, March 2017)
- 13,441,711 .deb Packages Download (July 2017)
- 18,839 Wiki page (July 2017)

Developers, Users

APP

ROS

ROBOT, SENSOR

Robot, Sensor Manufacturer

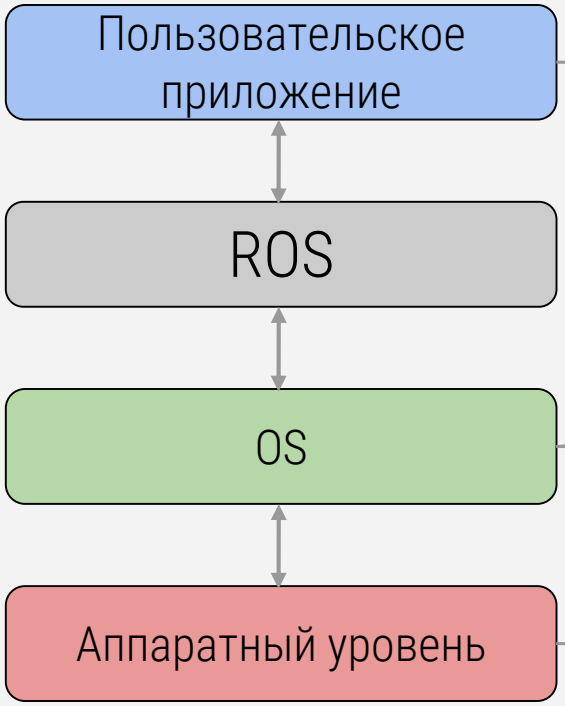
Applications	fetch beer, elevator ···
Simulators	gazebo, player/stage, STDR Simulator ···
Intelligent Modules	navigation, action, grasping ···
Libraries	tf, PCL, OpenCV, OpenRave ···
Device Drivers	camera_drivers, urg_node ···
Debugging Tools	rviz, rqt_graph, rosbag, rostopic ···
Message Passing	rosmaster, rosmsg, rosservice ···
Execution Tools	rosrun, roslaunch ···
Compile Tools	catkin_make, rosbuild ···
File Systems	roscd, rosls ···
Installer	rosinstall ···
Programming Languages	C++, Python, Lisp, Java, Ruby, MATLAB, etc

- Supports more than 90 robots and 80 sensors



Источник: ROS Robot Programming. YoonSeok Pyo, HanCheol Cho, RyuWoon Jung, TaeHoon Lim

МЕТА-ОПЕРАЦИОННАЯ СИСТЕМА



Мобильное, Web, Desktop приложение



Функции и сервисы предоставляемые роботом











Сенсоры

Актуаторы

Робо-платформы

КОМПОНЕНТЫ ROS



Источник: ROS Robot Programming. YoonSeok Pyo, HanCheol Cho, RyuWoon Jung, TaeHoon Lim

ИСТОРИЯ ROS

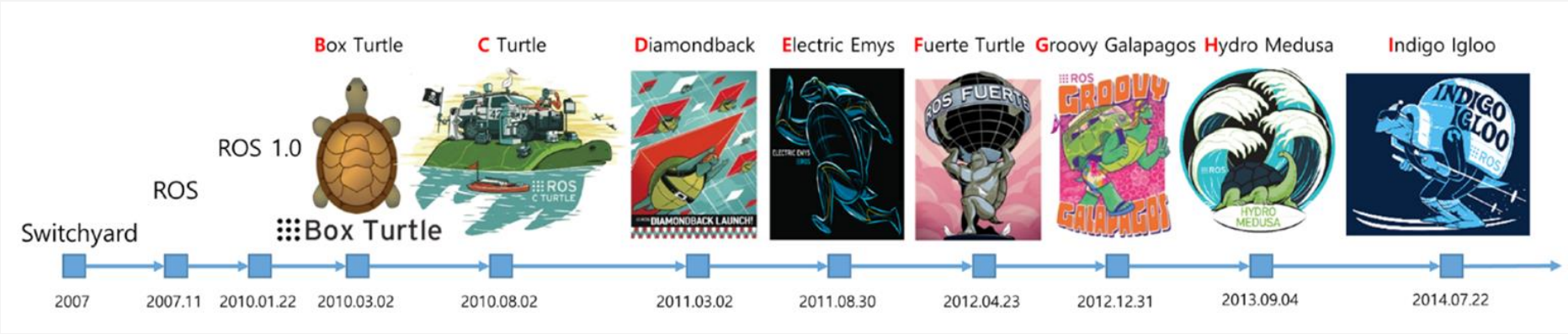
- 2007, Switchyard в Stanford. Еще до появления ROS в Стэнфорде было разработано несколько прототипов фреймворков для роботов. Для экспериментов использовались STanford Artificial Intelligence Robot (STAIR) и Personal Robotics (PR) program.
- 2007, Willow Garage — робототехнический инкубатор поддерживает разработку ROS.
- 2010, ROS 1.0

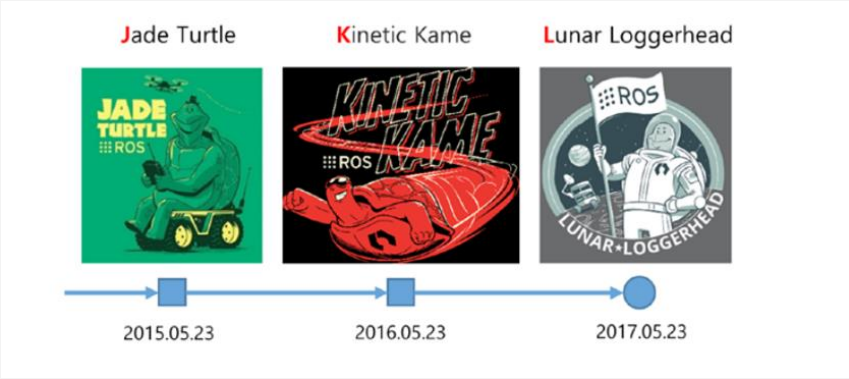


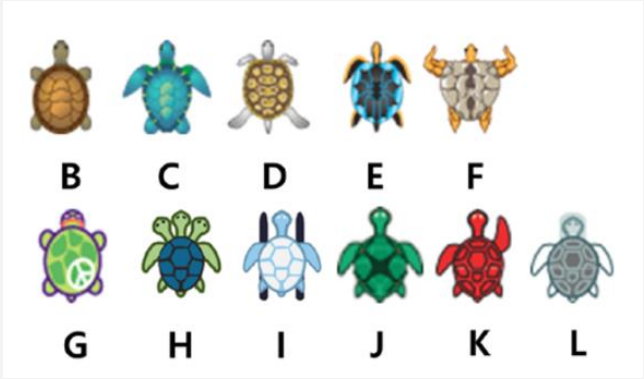
Источник: http://www.willowgarage.com/

ДИСТРИБУТИВЫ ROS

http://wiki.ros.org/Distributions





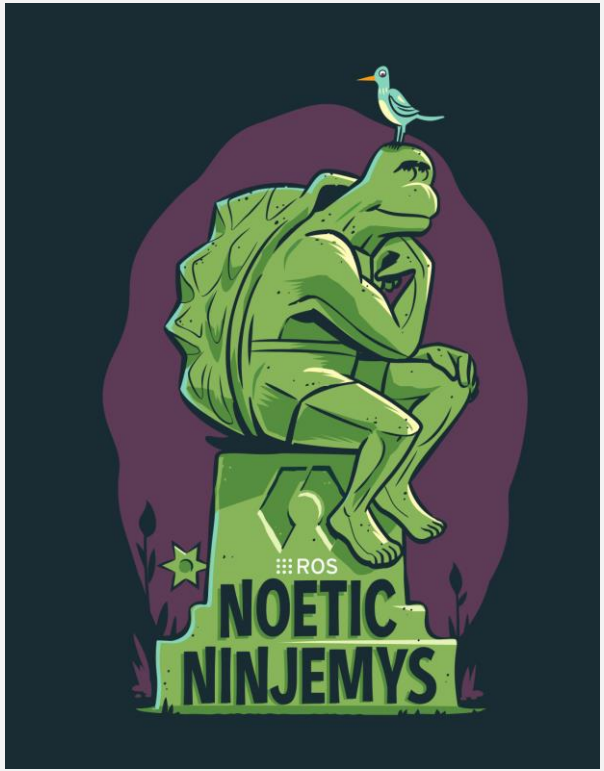


ДИСТРИБУТИВЫ ROS: Melodic



- Ubuntu 18.04 LTS «Bionic Beaver»
- Выпущена в мае 2018
- Поддерживается (как и Ubuntu 18.04) до мая 2023 года

ДИСТРИБУТИВЫ ROS: Noetic (рекомендуется к использованию)



- Ubuntu 20.04 LTS
- Поддержка python 3
- Выпущена в мае 2020
- Поддерживается до мая 2025 года

ROS 2

https://design.ros2.org/articles/why_ros2.html
http://docs.ros.org/en/rolling/

Почему ROS 2?

- Новые задачи, которые не стояли перед разработчиками ROS 1:
  - Группы роботов
  - Встраиваемые платформы
  - Системы реального времени
  - Проблемы передачи данных
  - Использование в коммерческих продуктах
  - Защита данных
  - …
- Появление новых технологий
- Чтобы учесть прошлые ошибки
- …

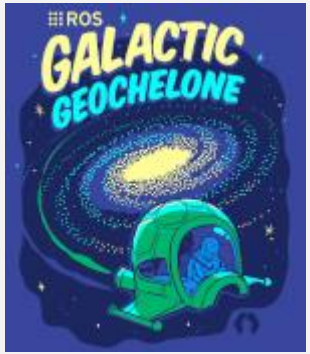
Distro	Release date	Logo	EOL date
Galactic Geochelone	May 23rd, 2021		November 2022
Foxy Fitzroy	June 5th, 2020		May 2023
Eloquent Elusor	November 22nd, 2019		November 2020
Dashing Diademata	May 31st, 2019		May 2021
Crystal Clemmys	December 14th, 2018		December 2019
Bouncy Bolson	July 2nd, 2018		July 2019
Ardent Apalone	December 8th, 2017		December 2018

ROS 2: Foxy



- Ubuntu 20.04 (Focal): amd64 and arm64
- Mac macOS 10.14 (Mojave)
- Windows 10 (Visual Studio 2019)

ROS 2: Galactic



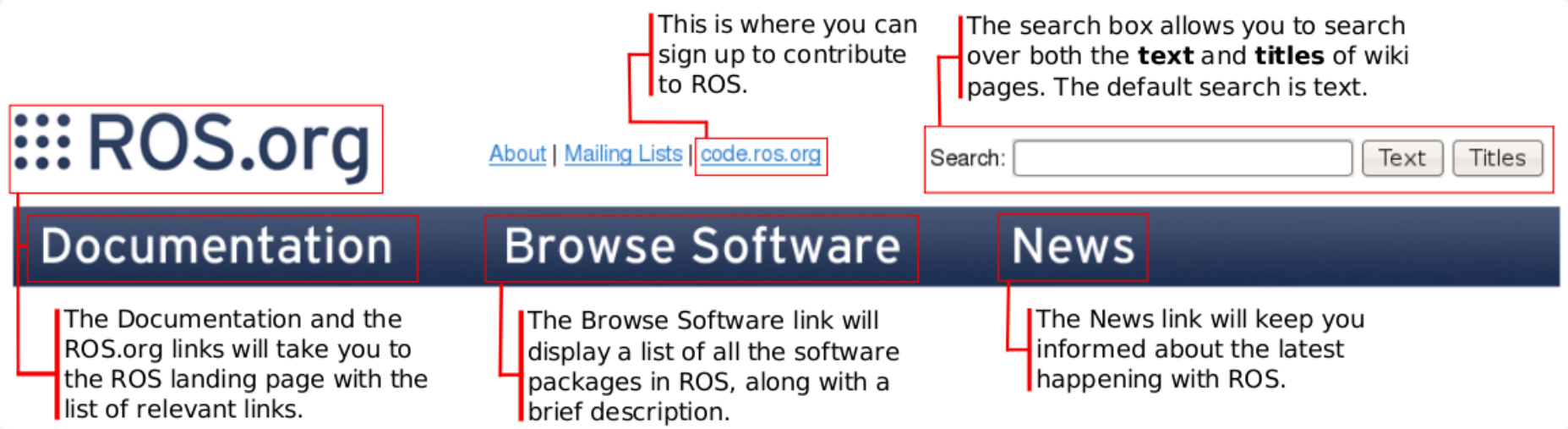
- Ubuntu 20.04 (Focal): amd64 and arm64;
- Windows 10 (Visual Studio 2019): amd64

ROS2 Tutorial (Линар Абдразаков)

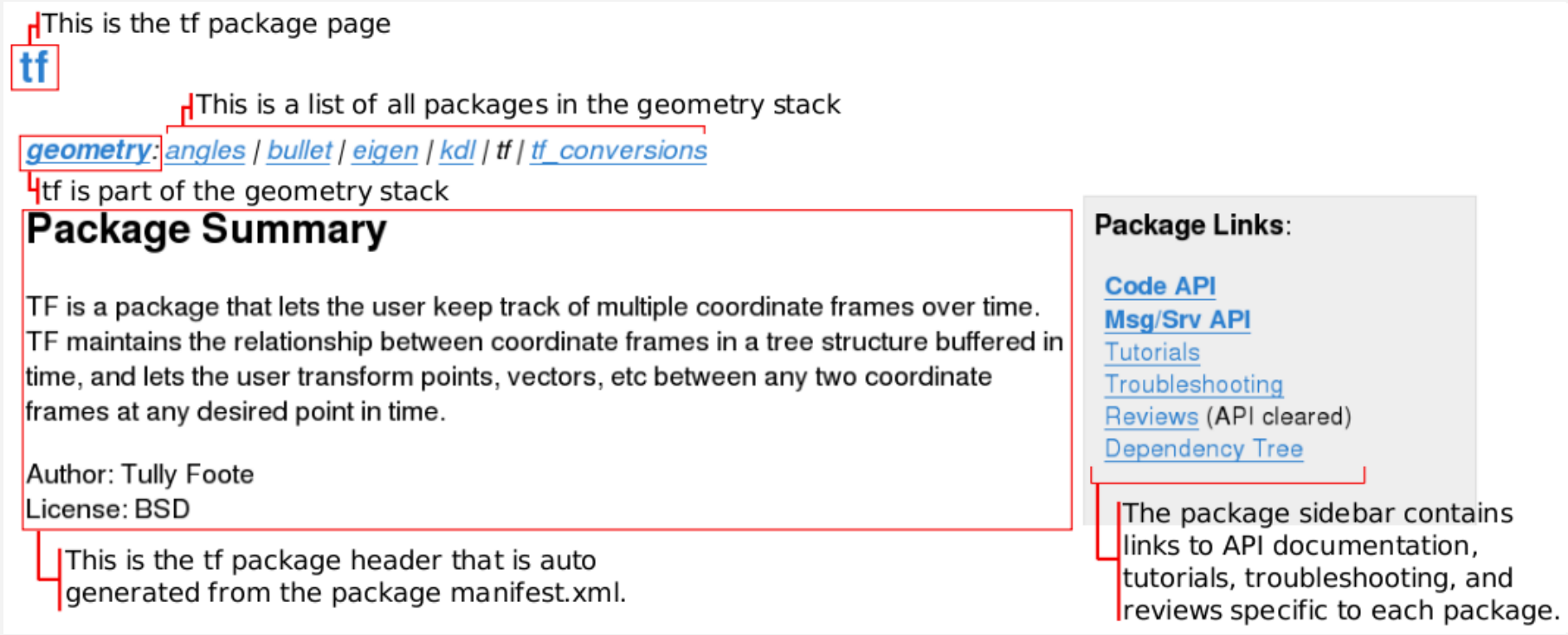
НАВИГАЦИЯ НА САЙТЕ С ДОКУМЕНТАЦИЕЙ ПО ROS

ROS1: http://wiki.ros.org/

ROS2: http://docs.ros.org/en/rolling/

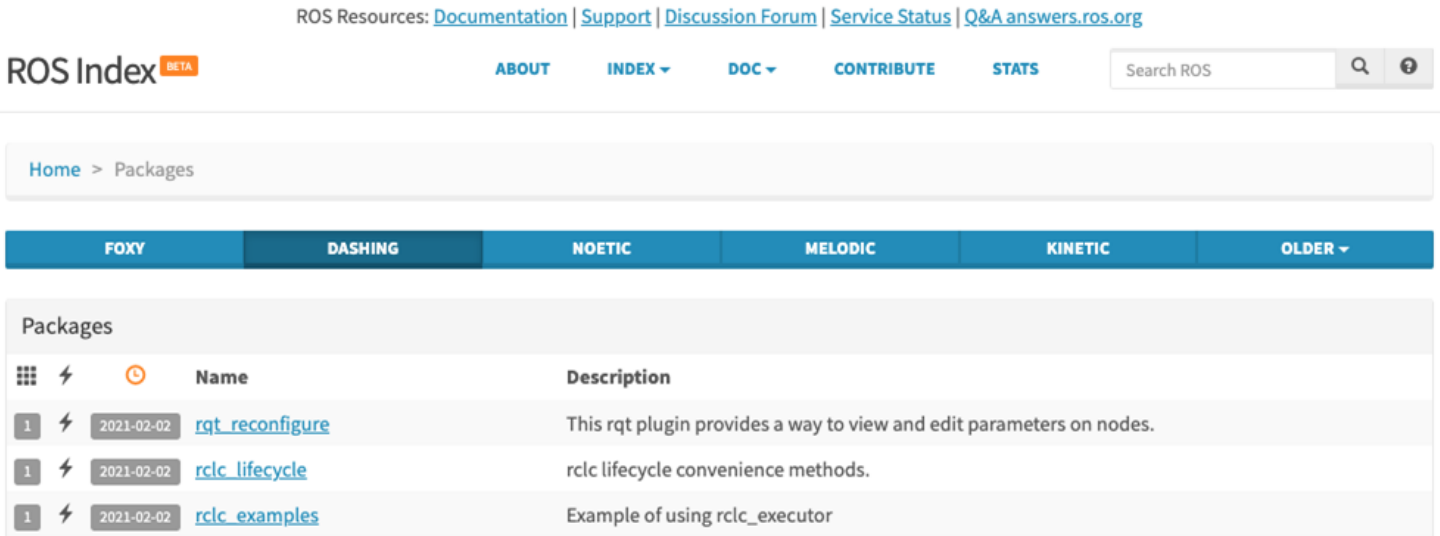


НАВИГАЦИЯ НА САЙТЕ ROS



Вступление

За последнее десятилетие **операционная система роботов (Robot Operating System, ROS)** выросла в несколько раз. В настоящий момент он имеет огромный список пакетов, где каждый пакет решает проблему либо частично, либо полностью, что исключает концепцию изобретения колеса заново. Созданные сообществом пакеты привели к тому, что был создан совершенно новый подход к робототехнике и предложена интеллектуальная начинка для существующих систем.



УСТАНОВКА ROS

https://www.ros.org/
http://wiki.ros.org/noetic/Installation/Ubuntu

Воспользуйтесь инструкцией: https://github.com/cds-mipt/raai_summer_school_cv_2021/tree/main/ros_basics

1. Добавление http://packages.ros.org в список “источников” из которых могут быть скачаны пакеты (в source.list)
2. Добавление ключей безопасности
3. Установка пакетов:
   - **Desktop-Full Install:** ROS, rqt, rviz, robot-generic libraries, 2D/3D симуляторы и 2D/3D пакеты обработки сенсорных данных
     - $ sudo apt install ros-<имя дистрибутива>-desktop-full
   - **Desktop Install:** ROS, rqt, rviz, и robot-generic libraries
     - $ sudo apt install ros-<имя дистрибутива>-desktop
   - **ROS-Base:** ROS package, build, и communication libraries. Без графических инструментов
     - $ sudo apt install ros-<имя дистрибутива>-base
   - **Отдельный пакет**
     - $ sudo apt install ros-<имя дистрибутива>-<имя пакета>

ЗАПУСК DOCKER КОНТЕЙНЕРА С ROS

http://wiki.ros.org/docker/Tutorials/Docker
https://hub.docker.com/_/ros

1. **Создаем Dockerfile** и описываем в нем установку и настройку необходимых пакетов
2. **Создаем** из Dockerfile **образ** (Image):
   - ❏ $ docker build --tag <имя образа> .
1. **Запускаем контейнер** из созданного образа:
   - ❏ $ docker run -it <имя образа>

Чтобы получить доступ к экрану из контейнера (необходимо для запуска графических приложений из контейнера), можно запустить следующим образом:

$ docker run -e DISPLAY=unix$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix -it <имя образа>

- ❏ Если при запуске возникает ошибка вида:
  No protocol specified
  Error: cannot open display: unix:0.0
- ❏ Необходимо разрешить локальные несетевые подключения на компьютере-хосте:
  $ xhost +local:

ОСНОВНЫЕ ТЕРМИНЫ

http://wiki.ros.org/ROS/Concepts

Нод – единица программного кода, исполняемая в отдельном потоке и выполняющий определенную вычислительную функцию. Ноды коммуницируют друг с другом через **топики**. Совокупность всех запущенных нодов образует **ROS граф**.

Топик – система передачи данных с механизмом публикации / подписки. Нод отправляет данные, публикуя их в топик, и читает, подписываясь на топик. Множество нодов может публиковать данные в один топик и читать данные из одного топика. Один нод может как подписываться, так и публиковать множество топиков.

Тип сообщения – описание формата сообщения, публикующегося в топике. Каждый топик публикует сообщения определенного типа.

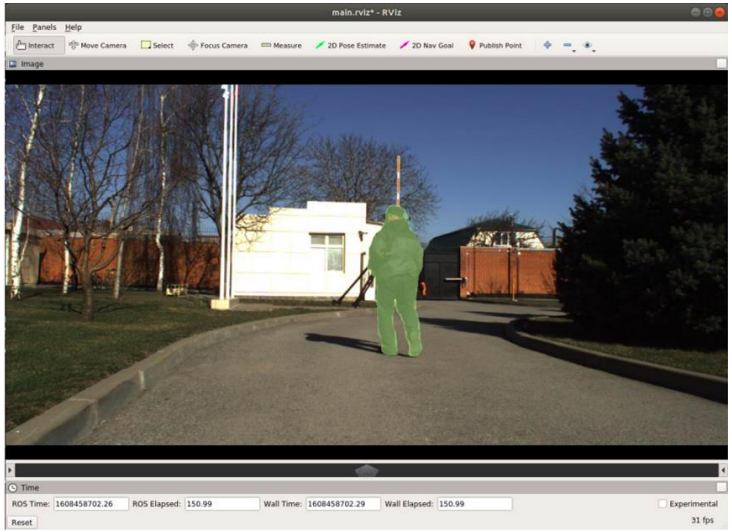
ROS граф – совокупность всех нодов и топиков, публикующихся в системе.

Пакет – единица организации ПО в ROS. Пакет может содержать программный код нодов, описание сообщений топиков, конфигурационные и другие файлы, связанные общим смыслом.

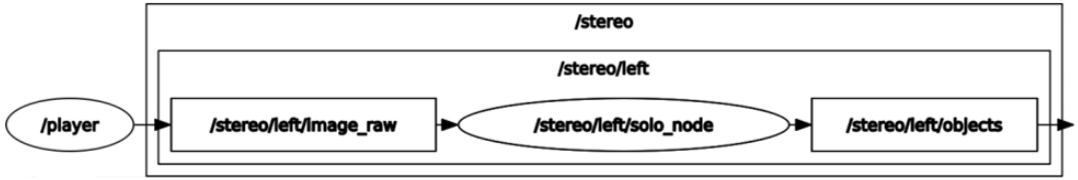
Инстанс-сегментация изображений с помощью нейросетевой модели SOLOv2

Компонент состоит из трех ROS-пакетов:

1. solo_ros - реализующий узел инстанс сегментации
2. camera_objects_msgs - с описанием формата сообщений
3. camera_objects_visualizer - реализующий визуализацию



- Компонент изолирован в docker-контейнере с Ubuntu 20.04 + ROS2 Foxy Fitzroy
- Успешно общается с хостом на Ubuntu 18.04 + ROS2 Dashing Diademanta
- conda окружение заменено на системное (pip3)



https://gitlab.com/sdbcs-nio3/itl_mipt/segm_tracking/alg/2d_segmentation/solov2/-/tree/feature/ros2_node
https://gitlab.com/sdbcs-nio3/itl_mipt/ros_common/camera_objects_msgs/-/tree/feature/ros2
https://gitlab.com/sdbcs-nio3/itl_mipt/ros_common/camera_objects_visualizer/-/tree/feature/ros2

SOURCE A.K.A СТРАШНЫЙ СОН НОВИЧКА

- Необходимый шаг перед **каждым** сеансом работы с ROS (выполняется в каждом вновь открытом терминале):

  ```
  $ source /opt/ros/<имя дистрибутива>/setup.bash
  ```

  В нашем случае <имя дистрибутива> = melodic

- Что делает $ source … ?
  - Устанавливает переменные окружения, необходимые для работы ROS
- Чтобы каждый раз не выполнять команду выше, ее часто добавляют в файл *~/.bashrc*, что приводит к ее автоматическому запуску для каждого нового терминала

  ```
  $ echo "source /opt/ros/<имя дистрибутива>/setup.bash" >> ~/.bashrc
  ```

SOURCE A.K.A СТРАШНЫЙ СОН НОВИЧКА

```
➜  ~ env | grep ROS
➜  ~ source /opt/ros/kinetic/setup.zsh
➜  ~ env | grep ROS
ROS_DISTRO=kinetic
ROS_ETC_DIR=/opt/ros/kinetic/etc/ros
ROS_PACKAGE_PATH=/opt/ros/kinetic/share
ROS_VERSION=1
ROS_ROOT=/opt/ros/kinetic/share/ros
ROS_MASTER_URI=http://localhost:11311
ROSLISP_PACKAGE_DIRECTORIES=
➜  ~
```

- ROS дистибутив
- Путь к установленным ROS-пакетам
- Версия ROS
- URI мастера
- ...

ЗАПУСК ROSCORE

http://wiki.ros.org/roscore

roscore — это набор нодов и программ, необходимых для запуска любого ROS-приложения. ***Roscore*** должен быть запущен для того, чтобы ноды могли коммуницировать. ***Roscore*** запускается командой:

- $ roscore

Также можно указать порт, на котором будет запущен ***master***:

- $ roscore -p 1234

roscore запускает:

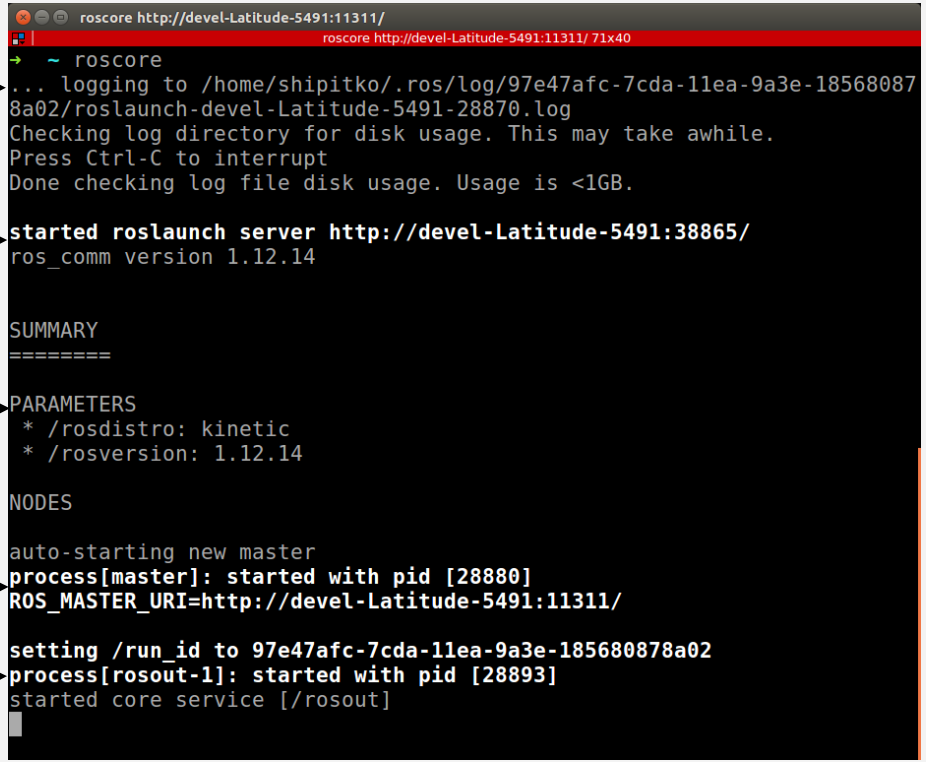
- rosmaster
- Parameter Server
- rosout — нод логирования

roscore всегда запускается автоматически, если запуск производится через ***roslaunch***, и ***roscore*** не был запущен до этого.

Ноды, которые запускаются при запуске ***roscore*** определены в ***roslaunch/roscore.xml*** (**внимание** не рекомендуется менять roscore.xml т.к. это повлияет на все последующие запуски ROS).

ЗАПУСК ROSCORE

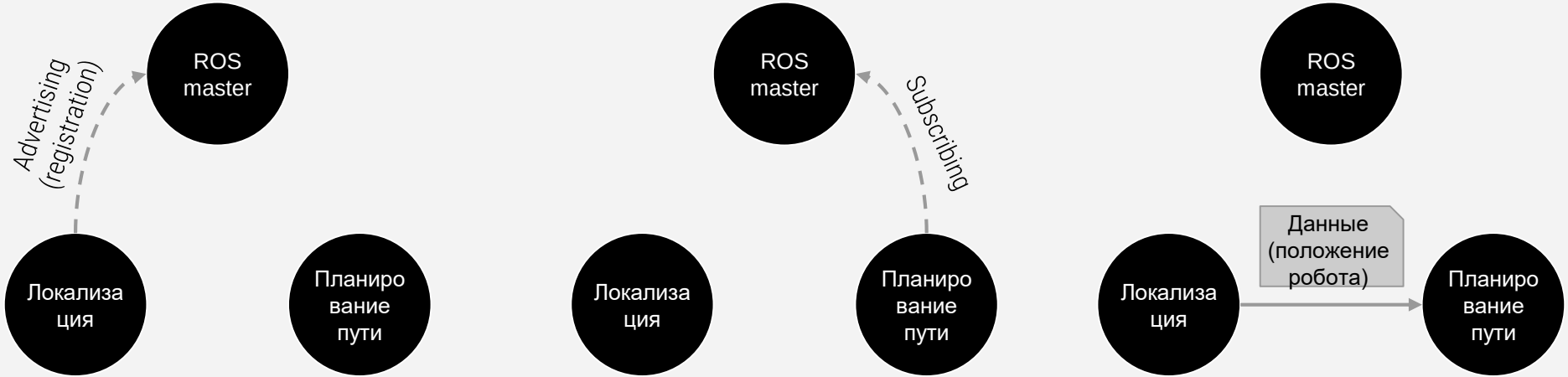
- Куда осуществляется логирование текущей сессия ROS
- Где запущен roslaunch server
- Какие параметры доступны на ***ROS Parameter Server***
- Где запущен ***rosmaster***
- Запуск нода ***rosout***



ROS MASTER

http://wiki.ros.org/Master

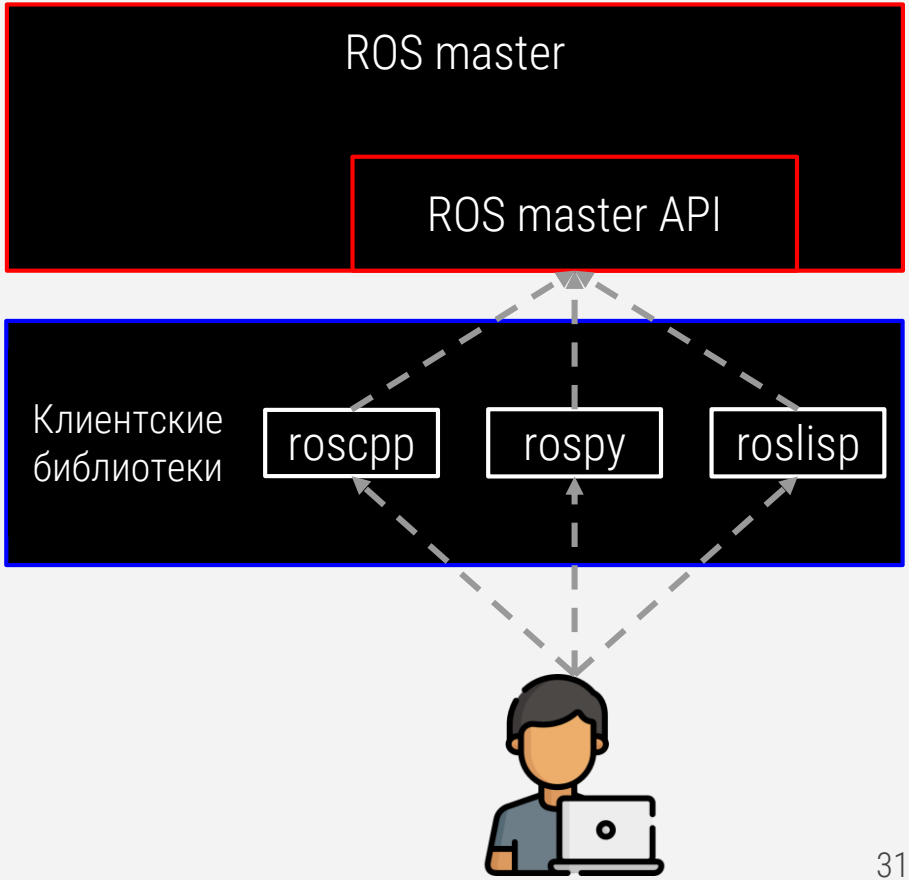
- Обеспечивает регистрацию топиков и сервисов. Можно сравнить с DNS-сервером — по имени топика / сервиса, предоставляет его URI.
- Предоставляет сервер параметров (***Parameter Server***)



ROS MASTER

http://wiki.ros.org/ROS/Master_API

- ***rosmaster*** предоставляет API для регистрации/отмены регистрации топиков и сервисов, а также для получения списка запущенных нодов, зарегистрированных топиков и т.д.
- ***Клиентские библиотеки*** используют ***rosmaster API*** и предоставляют разработчику простой механизм создания нодов, публикации топиков и т.д.
- **Важно!** Используйте ***rosmaster API*** напрямую, только если вы реализуете поддержку нового языка программирования. Список поддерживаемых языков можеты быть найден по ссылке: http://wiki.ros.org/Client%20Libraries



PARAMETER SERVER

http://wiki.ros.org/Parameter%20Server

Parameter Server – словарь параметров, доступный всем нодам в системе. Используется для хранения различных параметров и доступа к ним в режиме реального времени. *Запускается внутри* ***rosmaster***.

Параметры на сервере:

/camera/left/name: leftcamera
/camera/left/exposure: 1
/camera/right/name: rightcamera
/camera/right/exposure: 1.1

Запрос: /camera/left/name

Ответ:
leftcamera

Запрос: /camera/left

Ответ:
name: leftcamera
exposure: 1

Запрос: /camera

Ответ:

```
left: { name: leftcamera, exposure: 1 }
right: { name: rightcamera, exposure: 1.1 }
```

PARAMETER SERVER

http://wiki.ros.org/Parameter%20Server

Типы данных, поддерживаемые ***Parameter Server*** :

- ❏ 32-bit integers
- ❏ booleans
- ❏ strings
- ❏ doubles
- ❏ iso8601 dates
- ❏ lists
- ❏ base64-encoded binary data

Доступ к параметрам осуществляется через клиентские библиотеки (***roscpp***, ***rospy***, ...), а также инструмент командной строки ***rosparam***. Оба способа будут рассмотрены в курсе позднее.

ROSOUT

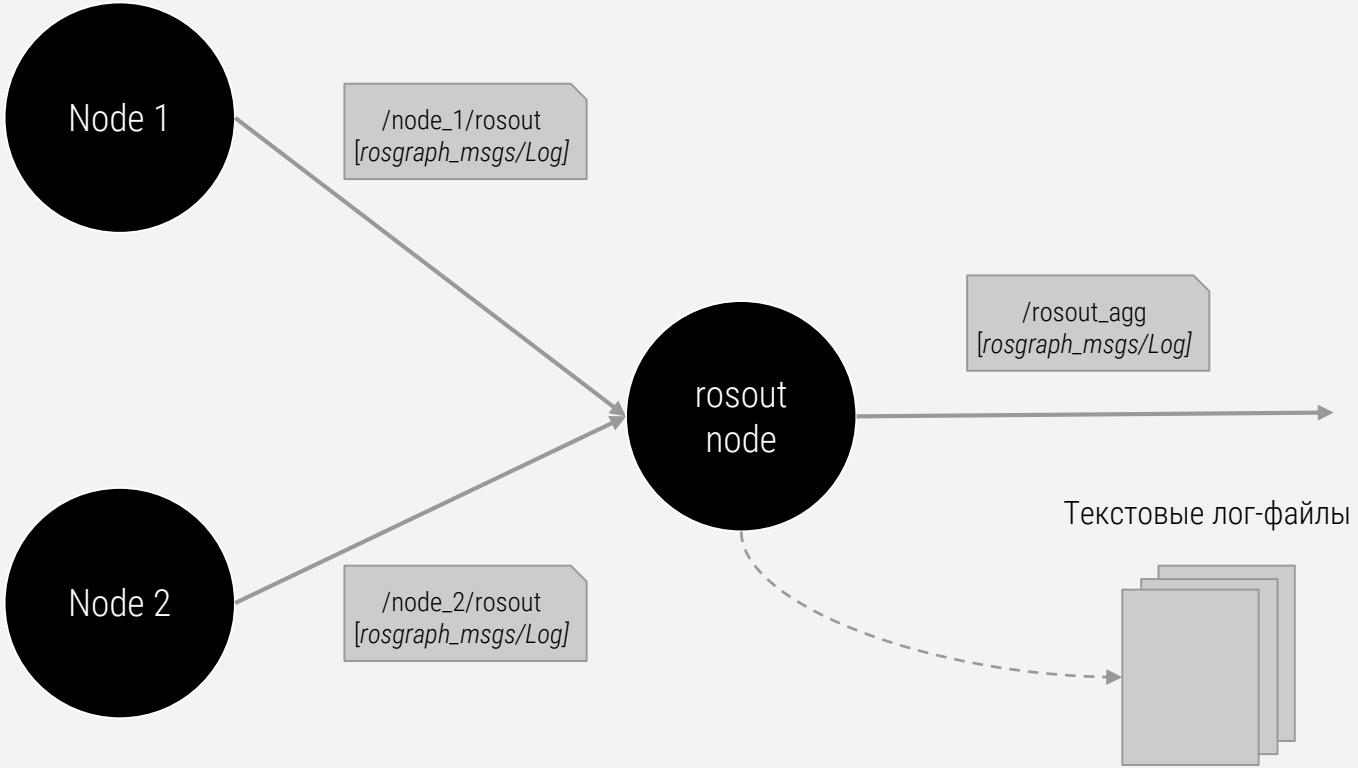
http://wiki.ros.org/rosout

Под термином ***rosout*** понимают несколько сущностей:

1. ***Нод rosout***. Подписывается на лог-сообщения каждого нода (<node namespace>/rosout), сохраняет их в текстовый лог-файл, а также дублирует в топик /rosout_agg.
2. ***Топик /rosout***. Стандартный топик для публикации лог-сообщений.
3. ***Топик /rosout_agg***. Содержит агрегированные лог-сообщения от всех нодов.
4. ***Тип сообщения rosgraph_msgs/Log***. Используется топиками /rosout и /rosout_agg.
5. ***API клиентских библиотек*** для работы с rosout логами в разных языках

ROSOUT

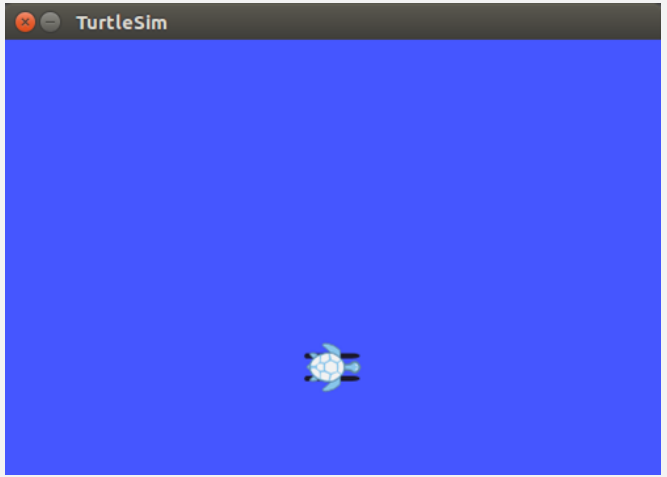
http://wiki.ros.org/rosout

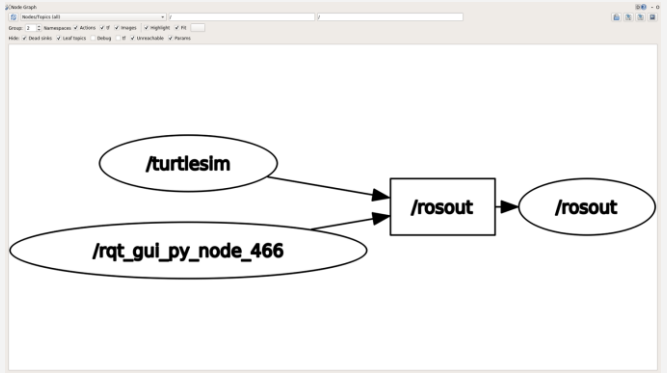


ROS “HELLO (TURTLE) WORLD!”

http://wiki.ros.org/ROS/Tutorials/UnderstandingNodes

- Устанавливаем переменные окружения
  $ source /opt/ros/<дистрибутив>/setup.bash
- Запускаем ***roscore*** в фоновом режиме
  $ roscore &
- Запускаем ***turtlesim_node*** из пакета ***turtlesim***
  $ rosrun turtlesim turtlesim_node
- Смотрим получившийся ***ROS граф***
  $ rqt_graph
- Запускаем ***turtle_teleop_key*** нод
  $ rosrun turtlesim turtle_teleop_key
- Управляем симуляцией нажатием “стрелок” и снова смотрим получившийся ***ROS граф***





ROS “HELLO (TURTLE) WORLD!”

http://wiki.ros.org/ROS/Tutorials/UnderstandingNodes

- Проверяем запущенные ноды и существующие топики

  $ rostopic list

  $ rosnode list

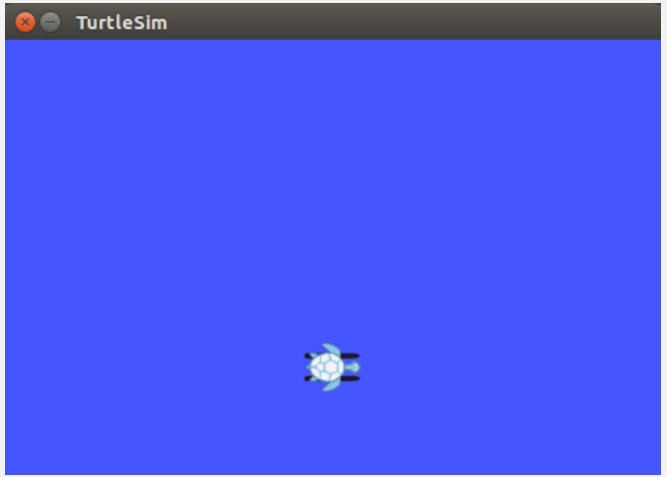
- Проверяем тип сообщения, публикуемого в топик ***/turtle1/pose***

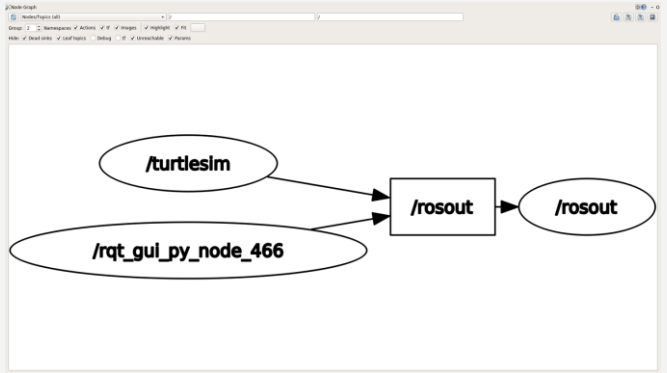
  $ rostopic info /turtle1/pose

  $ rosmsg info turtlesim/Pose

- Смотрим на данные, публикуемые в топик ***/turtle1/pose***

  $ rostopic echo /turtle1/pose





Проблемы в ROS1

1) Первое ограничение состоит в том, что ROS1 не поддерживает концепцию мультимастера.
2) Второе ограничение ROS1 заключается в том, что, несмотря на тот факт, что ноды могут без проблем общаться по сети между собой, нет безопасного пути в этом общении. Любой, кто имеет возможность подключиться к мастеру (master-node), может получить доступ к топикам и далее либо эксплуатировать или изменять их. Поэтому основное предназначение ROS1 — проверка концепции или прототипирование для научных целей.
3) И третье основное ограничение ROS1 — данная система не является real-time системой. Что приводит к задержкам в получении данных и их потере.

   Все вышеизложенные недостатки подтолкнули к созданию ROS2

zoldaten. ROS2 vs ROS1.Установка ROS2 на Ubuntu 18.04. URL: https://habr.com/ru/post/492058/

Преимущества ROS2

1) позволяет устанавливать безопасные соединения между компонентами системы (узлами)
2) взаимодействие осуществляется в режиме «real-time»
3) соединение нескольких роботов в одну сеть упрощено
4) улучшено «качество общения» между узлами
5) реализуется уровень ROS архитектуры непосредственно на аппаратном обеспечении
6) используются новейшие пакеты обновлений (ROS1 обновляется не так часто).

zoldaten. ROS2 vs ROS1.Установка ROS2 на Ubuntu 18.04. URL: https://habr.com/ru/post/492058/

Преимущества ROS2

ROS2 использует те же принципы работы, что и ROS1. В то же время, ROS2 не является частью ROS1. ROS2 был создан, чтобы перешагнуть ограничения путем использования более новых зависимостей и инструментов.

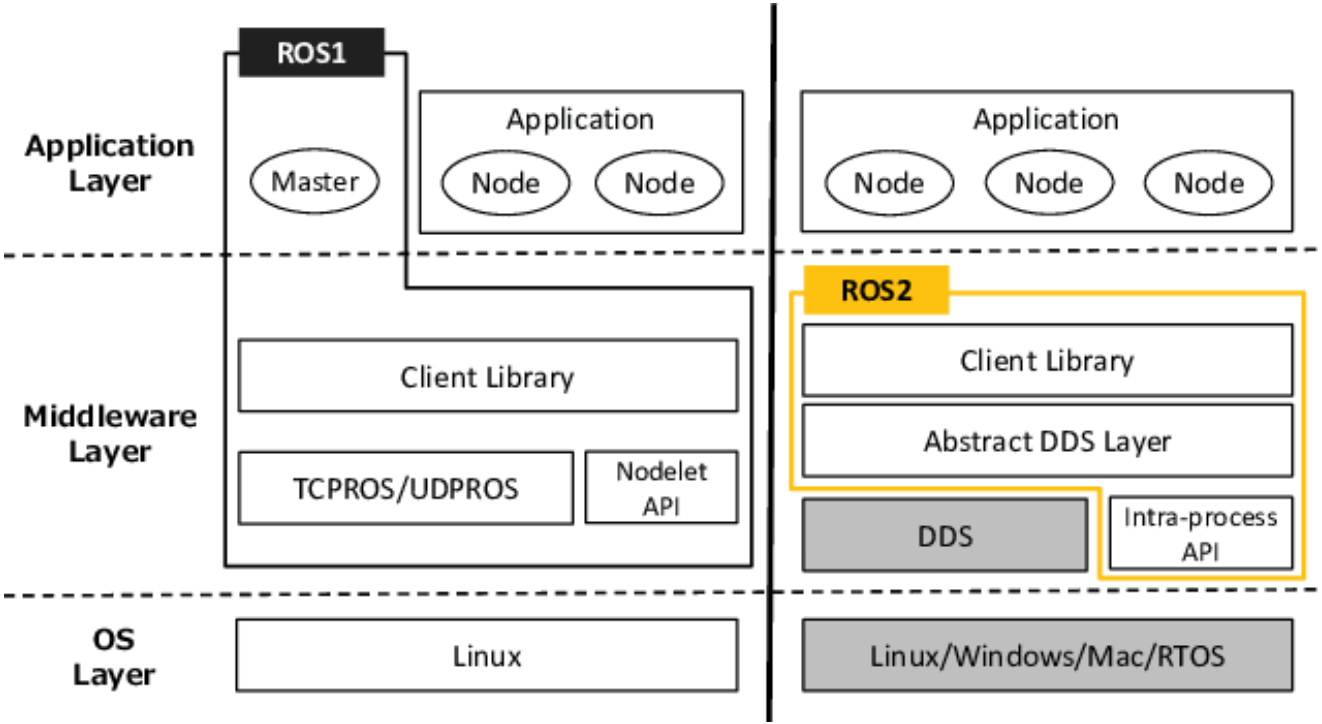
В отличие от ROS1, стек которого написан на C++, а клиентские библиотеки на C++ и Python, в ROS-2 также используется язык C. Представлен независимый уровень, написанный на C, который взаимодействует с клиентскими библиотеками ROS2, такими как rclcpp, rclpy, and rcljava.

ROS2 лучше поддерживает сетевые конфигурации и реализует надежные соединения.

ROS2 также более не использует концепцию nodelets (http://wiki.ros.org/nodelet) и поддерживает возможность инициализации нескольких нод одновременно (multinode initialization). В отличие от ROS1, еще одной отличительной чертой ROS2 является возможность отслеживать статус ноды и получать сообщения, если нода или топик добавились или были удалены. Это можно использовать для создания отказоустойчивых систем.

zoldaten. ROS2 vs ROS1.Установка ROS2 на Ubuntu 18.04. URL: https://habr.com/ru/post/492058/

Различия между ROS1 и ROS2



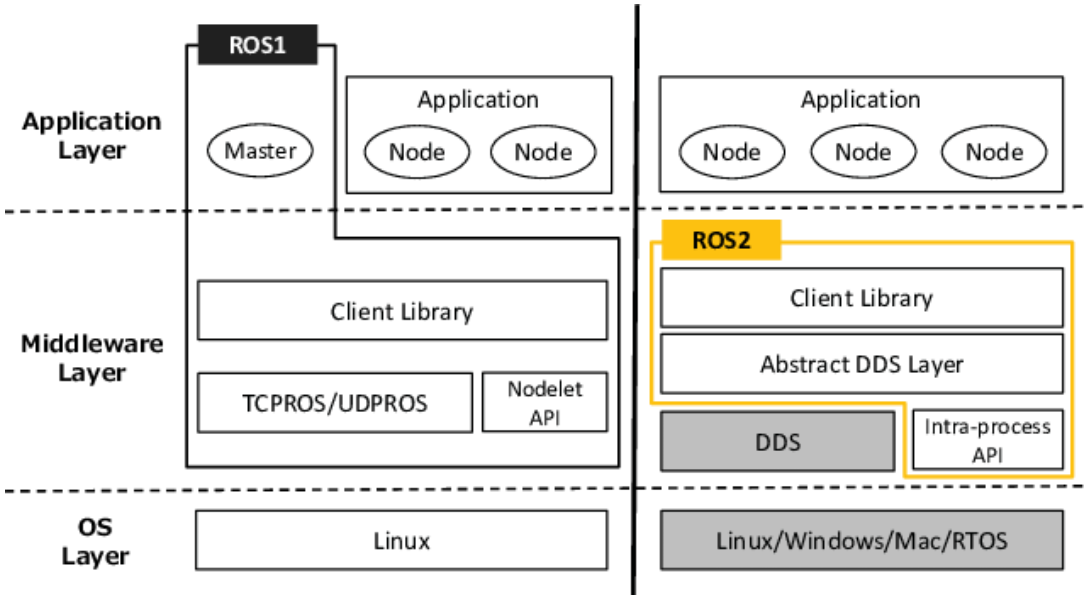
Maruyama Y., Kato S., Azumi T. Exploring the performance of ROS2 //Proceedings of the 13th International Conference on Embedded Software. – 2016. – С. 1-10. URL: https://www.researchgate.net/publication/309128426_Exploring_the_performance_of_ROS2

Различия между ROS1 и ROS2

Промежуточный уровень (Middleware Layer)

В ROS 1 используется пользовательский формат сериализации, пользовательский транспортный протокол, а также пользовательский центральный механизм обнаружения.

ROS 2 имеет абстрактный интерфейс промежуточного программного обеспечения, через который осуществляется сериализация, транспортировка и обнаружение.



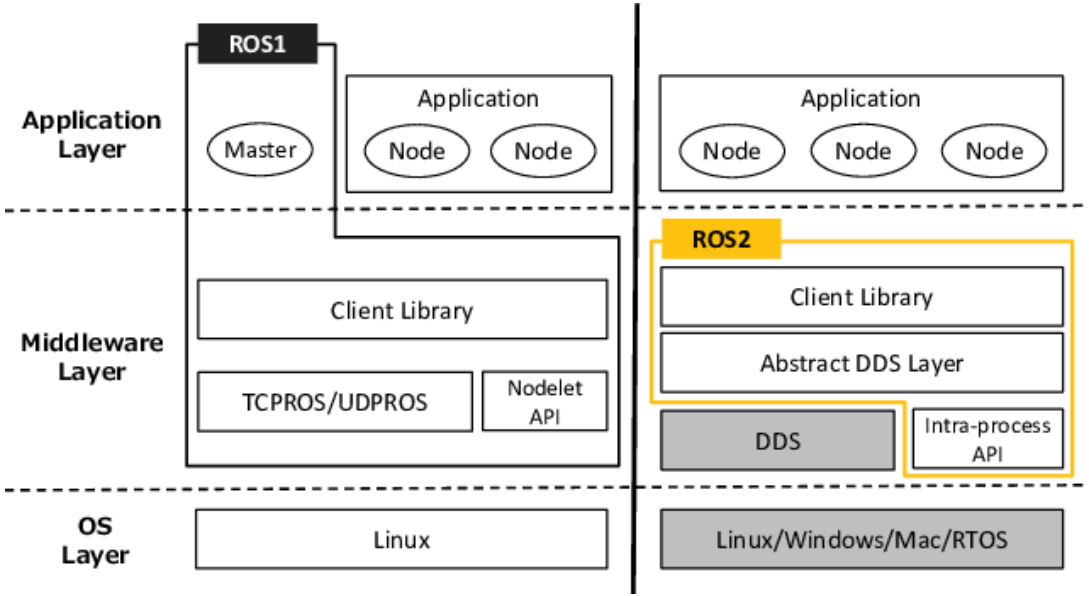
Maruyama Y., Kato S., Azumi T. Exploring the performance of ROS2 //Proceedings of the 13th International Conference on Embedded Software. – 2016. – С. 1-10. URL: https://www.researchgate.net/publication/309128426_Exploring_the_performance_of_ROS2

Различия между ROS1 и ROS2

ROS Master

ROS 1 - это централизованная система, в которой главный узел всегда необходим для работы других узлов. Этот ведущий узел должен быть выполнен до запуска любой другой программы ROS. Команда roscore запускает этот ведущий узел.

В ROS 2 этот ведущий узел НЕ нужен. Все управляется этим промежуточным уровнем, который в настоящее время базируется на DDS (Data Distribution Service).



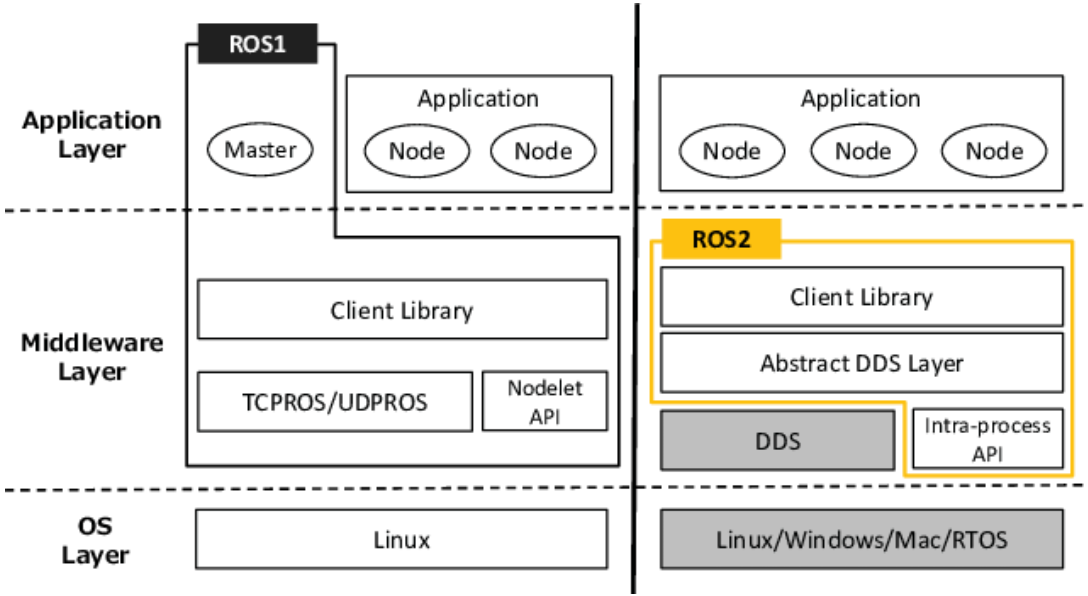
Maruyama Y., Kato S., Azumi T. Exploring the performance of ROS2 //Proceedings of the 13th International Conference on Embedded Software. – 2016. – С. 1-10. URL: https://www.researchgate.net/publication/309128426_Exploring_the_performance_of_ROS2

Различия между ROS1 и ROS2

Поддерживаемые ОС

ROS 1 поддерживается только на системах Linux.

ROS 2 поддерживается на системах Linux, Mac OC и Windows.



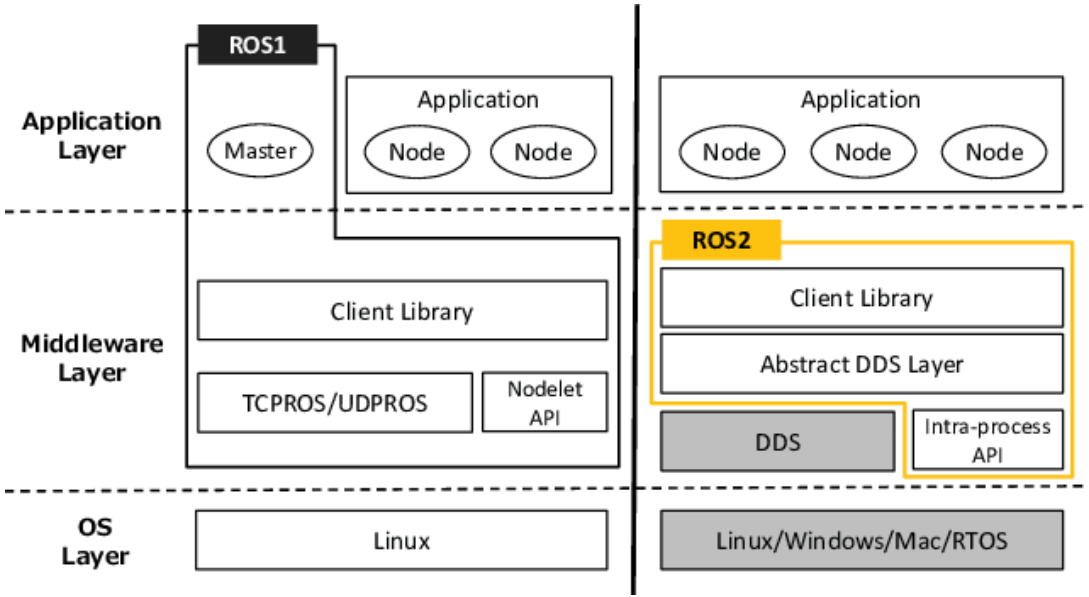
Maruyama Y., Kato S., Azumi T. Exploring the performance of ROS2 //Proceedings of the 13th International Conference on Embedded Software. – 2016. – С. 1-10. URL: https://www.researchgate.net/publication/309128426_Exploring_the_performance_of_ROS2

Различия между ROS1 и ROS2

Языки программирования

Ядро ROS 1 нацелено на C++03 и не использует возможности C++11 в своем API. В ROS 2 широко используется C++11 и некоторые части из C++14. В будущем ROS 2 может начать использовать C++17 при условии, что он поддерживается на всех основных платформах.

ROS 1 нацелена на Python 2 (Только в версии ROS1 Noetic появилась поддержка Python 3).
Для ROS 2 требуется как минимум Python версии 3.5.



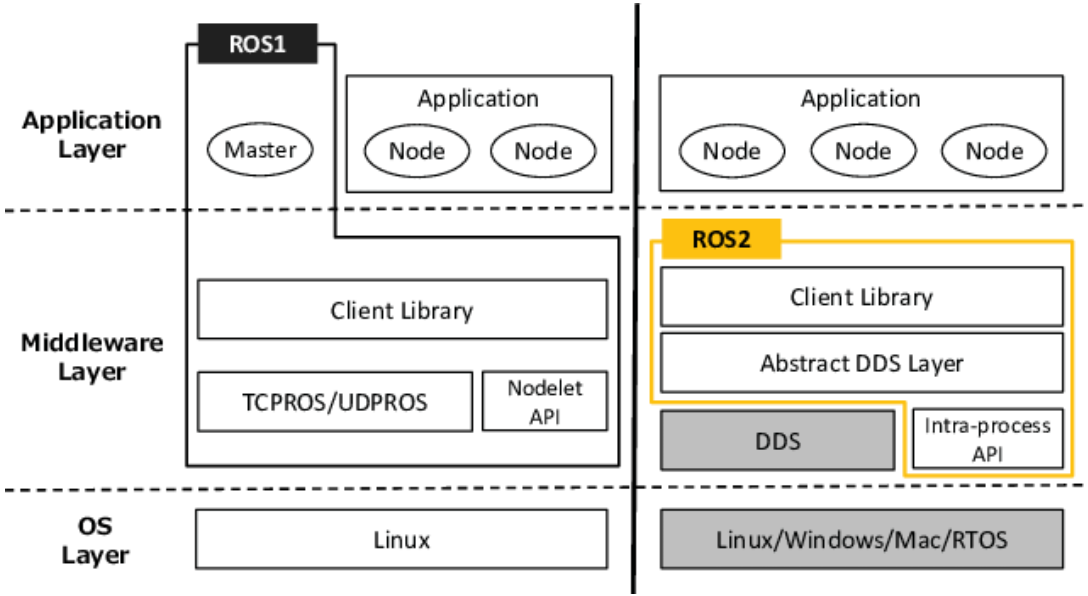
Maruyama Y., Kato S., Azumi T. Exploring the performance of ROS2 //Proceedings of the 13th International Conference on Embedded Software. – 2016. – С. 1-10. URL: https://www.researchgate.net/publication/309128426_Exploring_the_performance_of_ROS2

Различия между ROS1 и ROS2

Несколько узлов

В ROS1 невозможно создать более одного узла в процессе.

В ROS2 можно создавать несколько узлов в процессе.



Maruyama Y., Kato S., Azumi T. Exploring the performance of ROS2 //Proceedings of the 13th International Conference on Embedded Software. – 2016. – С. 1-10. URL: https://www.researchgate.net/publication/309128426_Exploring_the_performance_of_ROS2

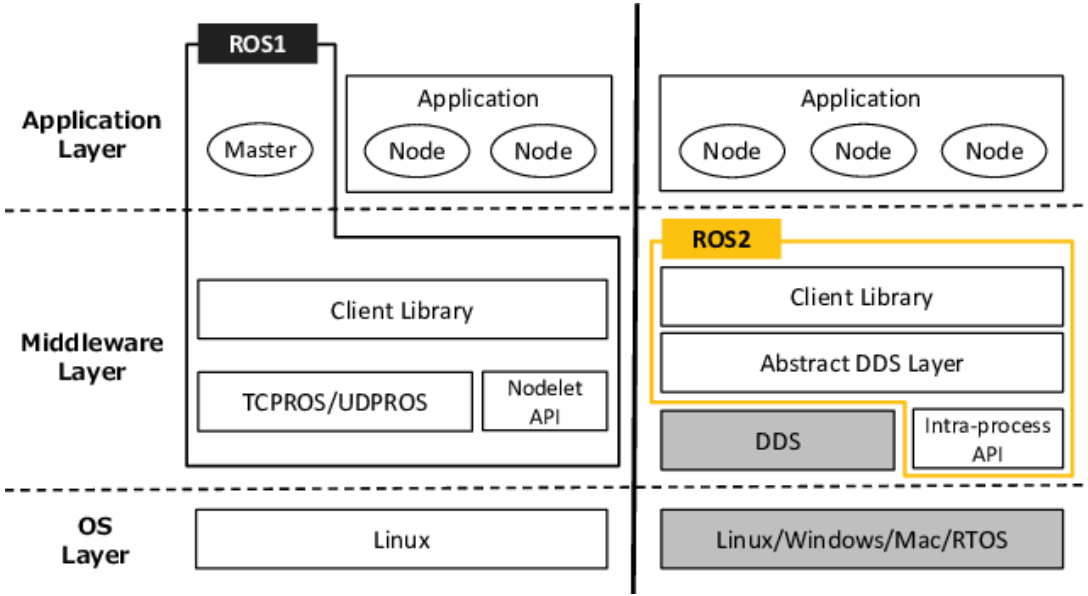
Различия между ROS1 и ROS2

roslaunch

Launch-файлы очень распространены в ROS как среди пользователей, так и среди разработчиков. Они обеспечивают удобный способ запуска нескольких узлов и мастера, а также другие требования к инициализации, такие как настройка параметров.

В ROS1 launch-файлы определяются в XML с очень ограниченными возможностями.

В ROS2 launch-файлы написаны на Python, что позволяет использовать более сложную логику, например, условия и т.д.



Maruyama Y., Kato S., Azumi T. Exploring the performance of ROS2 //Proceedings of the 13th International Conference on Embedded Software. – 2016. – С. 1-10. URL: https://www.researchgate.net/publication/309128426_Exploring_the_performance_of_ROS2

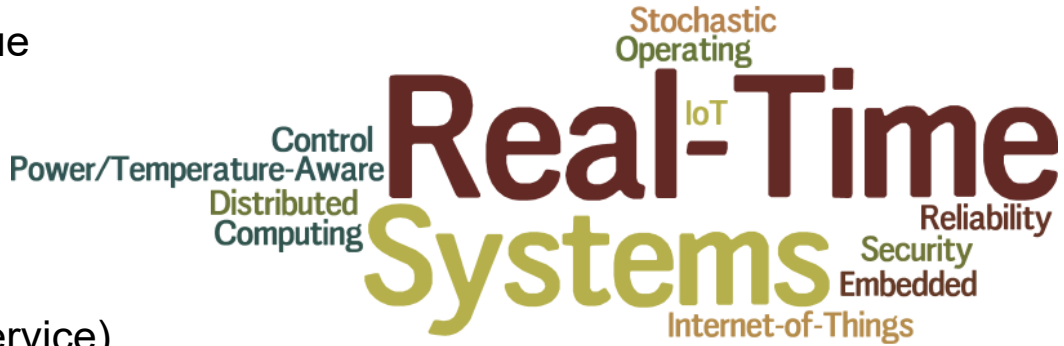
Недостатки ROS2

- Большая часть open-source решений разработана на ROS1 и их использование с ROS2 требует портирования (возможно использование ros1_bridge)
- Не все привычные стандартные пакеты портированы на ROS2 (например функционал проигрывания bag файлов существенно урезан)
- Система запуска python узлов фактически делает невозможным использование conda окружений (rclpy не распространяется как conda или pypi пакет, это apt пакет)



Системы реального времени

- Детерминированность - для одних и тех же входных данных система производит одинаковые выходные данные
- Точки завершения (deadline) - промежуток времени, за которое система должна дать ответ
- Параметры качества (quality of service) - характеристики сети (пропускная способность, задержки, устойчивость к ошибкам и т.д.)



https://design.ros2.org/articles/realtime_background.html

Поддержка систем real-time в ROS2 - QoS

Использование DDS в качестве механизма обмена сообщениями позволяет гибко настраивать параметры сети (QoS) в диапазоне:



При этом DDS использует только UDP и самостоятельно обеспечивает требуемый уровень качества соединения.

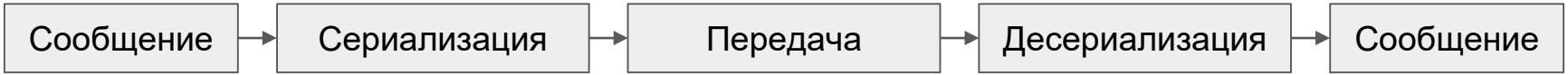
queue_size - пример QoS параметра, пришедший из ROS1

Параметры сети указываются при создании подключения к топику

https://index.ros.org/doc/ros2/Tutorials/Real-Time-Programming/
https://index.ros.org/doc/ros2/Concepts/About-Quality-of-Service-Settings/

Несколько узлов в одном процессе (C++)

Мотивация: если заранее известно, что узлы будут запускаться на одном физическом устройстве, то обмен сообщениями может быть ускорен



ROS1

- узел оформляется в виде nodelet
- nodelet - узел компилируется в разделяемую библиотеку (*.so)
- в момент запуска несколько nodelet подгружаются в одном исполняемом файле (nodelet_manager) в разных потоках

ROS2

- узел оформляется как наследник Node
- объект класса Node может быть запущен либо как самостоятельный узел, либо как часть исполняемого файла
- zero-copy обмен достигается при использовании std::unique_ptr

Таким образом, решить как будет использован узел (standalone/nodelet) нужно перед началом разработки, либо иметь две реализации

Решение о формате использования узла откладывается на момент запуска

https://index.ros.org/doc/ros2/Tutorials/Composition/
https://index.ros.org/doc/ros2/Tutorials/Intra-Process-Communication/

Краткое руководство работы с ROS1 и ROS2

Перед использованием ROS1, необходимо выполнить

```
source /opt/ros/melodic/setup.bash
```

Запуск ROS1 узла

```
rosrun <package_name> <node_name>
```

Работа с топиками

```
rostopic list

rosmsg list

rosmsg show std_msgs/String

rostopic pub <topic> <msg_type> <msg>

rostopic echo <topic>
```

Перед использованием ROS2, необходимо выполнить

```
source /opt/ros/rolling/setup.bash
```

Запуск ROS2 узла

```
ros2 run <package_name> <node_name>
```

Работа с топиками

```
ros2 topic list

ros2 msg list

ros2 msg show std_msgs/String

ros2 topic pub <topic> <msg_type> <msg>

ros2 topic echo <topic>
```

Video-classes: https://www.youtube.com/watch?v=bxoh4xY7RHo&list=PLK0b4e05LnzYNBzqXNm9vFD9YXWp6honJ&index=9

ROS1 Tutorials: http://wiki.ros.org/ROS/Tutorials

ROS2 Tutorials: https://index.ros.org/doc/ros2/Tutorials/

Краткое руководство работы с ROS1 и ROS2

Создание ROS1 рабочей среды

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws
catkin_make
source ~/catkin_ws/devel/setup.bash
```

Создание ROS1 пакета на C++

```
cd catkin_ws/src

catkin_create_pkg beginner_tutorials roscpp

cd ~/catkin_ws

catkin_make
```

Создание ROS2 рабочей среды

```
mkdir -p ~/my_ros2_ws/src
cd ~/my_ros2_ws
colcon build
source ~/my_ros2_ws/install/setup.bash
```

Создание ROS2 пакета на C++

```
cd ~/my_ros2_ws/src

ros2 pkg create ros2_cpp_pkg --build-type \
ament_cmake --dependencies rclcpp

cd ~/my_ros2_ws/
colcon build --symlink-install
```

Video-classes: https://www.youtube.com/watch?v=bxoh4xY7RHo&list=PLK0b4e05LnzYNBzqXNm9vFD9YXWp6honJ&index=9

ROS1 Tutorials: http://wiki.ros.org/ROS/Tutorials

ROS2 Tutorials: https://index.ros.org/doc/ros2/Tutorials/

Краткое руководство работы с ROS1 и ROS2

Создание ROS1 пакета на Python

```
cd ~/catkin_ws/src

catkin_create_pkg beginner_tutorials rospy

cd ~/catkin_ws

catkin_make

source devel/setup.bash
```

Создание ROS2 пакета на Python

```
cd ~/my_ros2_ws/src

ros2 pkg create --build-type ament_python --
node-name my_node my_package --dependencies
rclpy

cd ~/my_ros2_ws

colcon build --packages-select my_package

source install/setup.bash
```

Video-classes: https://www.youtube.com/watch?v=bxoh4xY7RHo&list=PLK0b4e05LnzYNBzqXNm9vFD9YXWp6honJ&index=9
ROS1 Tutorials: http://wiki.ros.org/ROS/Tutorials ROS2 Tutorials:
https://index.ros.org/doc/ros2/Tutorials/

Запуск ros1_bridge

Перед запуском необходимо установить ros1_bridge

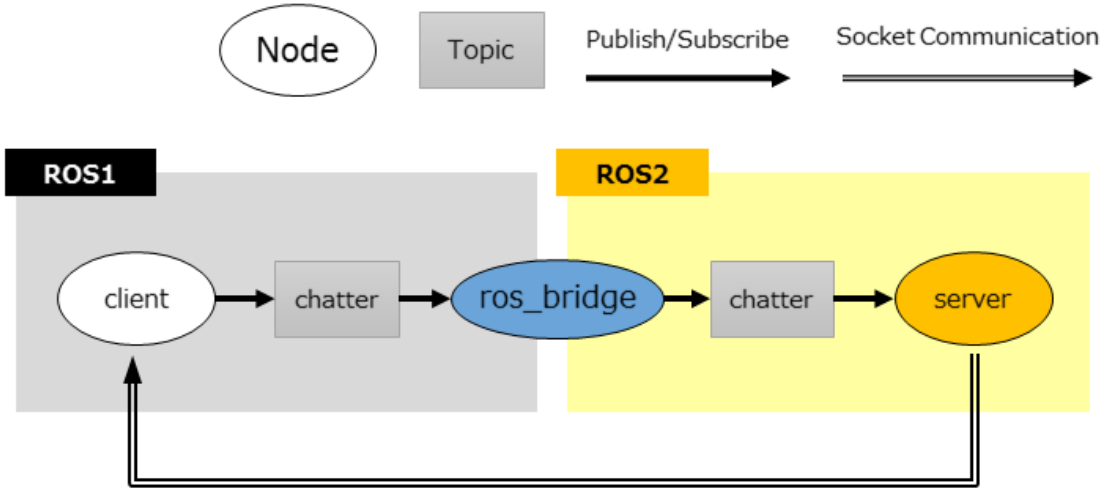
```
sudo apt install ros-dashing-ros1-bridge
```

Shell 1

```
source /opt/ros/melodic/setup.bash
roscore
```

Shell 2

```
source /opt/ros/melodic/setup.bash
source /opt/ros/dashing/setup.bash
export ROS_MASTER_URI=http://localhost:11311
ros2 run ros1_bridge dynamic_bridge
```



Lifecycled-узлы (Новая функция в ROS2)

Управляемые Lifecycled-узлы позволяют лучше контролировать состояние системы ROS2. Это позволяет roslaunch убедиться, что все компоненты были созданы правильно, прежде чем система позволит любому компоненту начать выполнять свое поведение. Это также позволяет перезапускать или заменять узлы в режиме онлайн.

Сейчас есть 4 основных состояния (Primary States):

- `Unconfigured`
- `Inactive`
- `Active`
- `Finalized`

Существует также 6 переходных состояний (Transition States), которые являются промежуточными состояниями во время запрашиваемого перехода.

- `Configuring`
- `CleaningUp`
- `ShuttingDown`
- `Activating`
- `Deactivating`
- `ErrorProcessing`

Lifecycled-узлы

Состояние узла может быть изменено с помощью lifecycle_service_client в соответствии с конечным автоматом состояний.

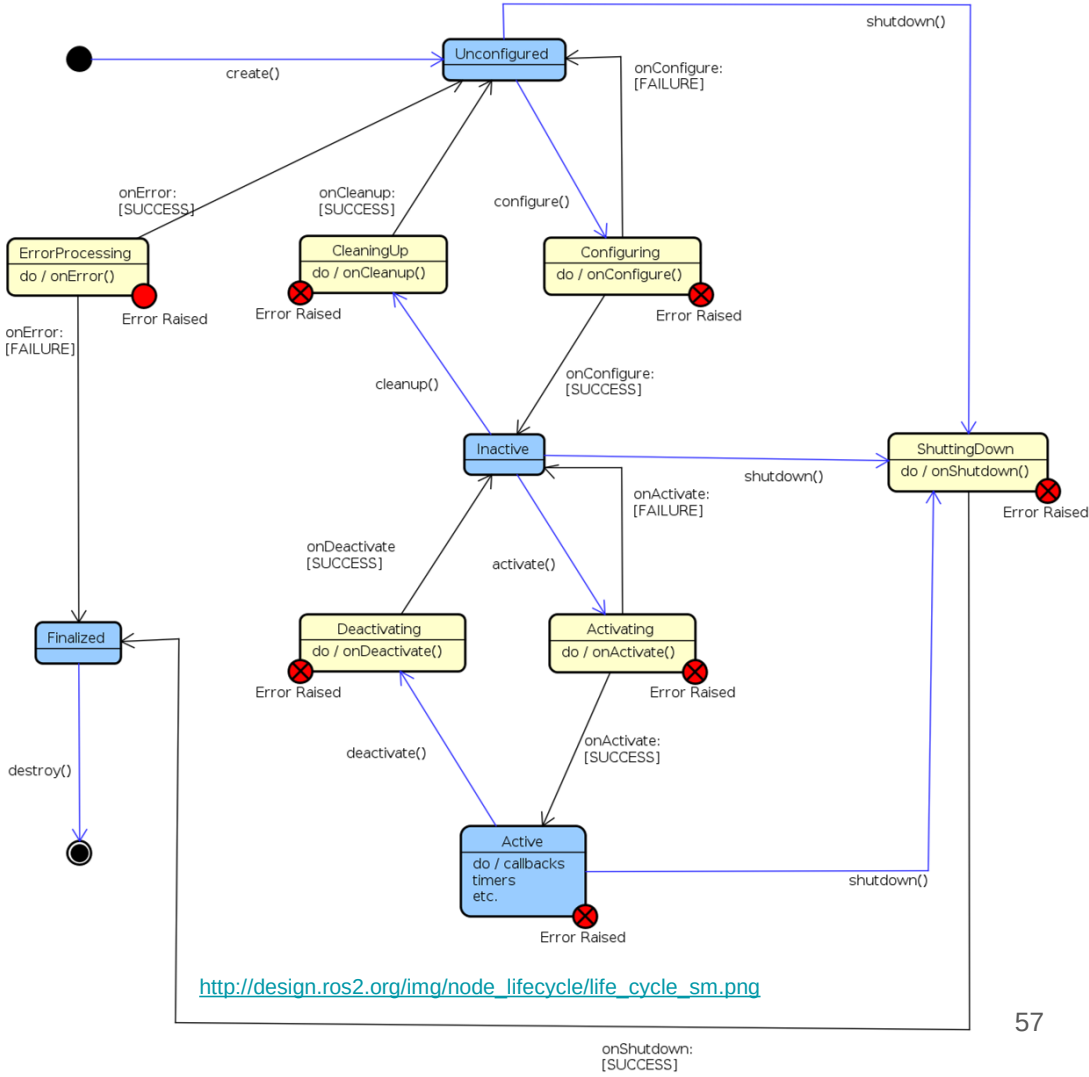
Чтобы реализовать Lifecycled-узел, узел должен быть унаследован(inherited) от объекта Life_Cycle_Node (а не от объекта Node)

Чтобы получить/выбрать состояние узла и просмотреть доступные состояния:

```
$ ros2 lifecycle get /node_name

$ ros2 lifecycle set /node_name configure

$ ros2 lifecycle list /node_name
```



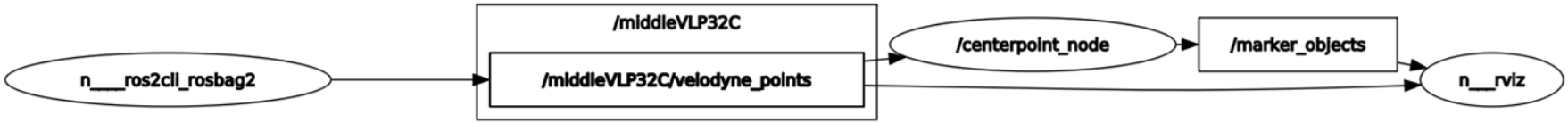
http://design.ros2.org/img/node_lifecycle/life_cycle_sm.png

Обнаружение и трекинг объектов в трехмерном облаке точек на основе нейросетевого подхода CenterPoint

Компонент состоит из двух ROS-пакетов:

1. centerpoint_node - реализует узел обнаружения и трекинга объектов
2. lidar_objects_msgs - пакет с описанием формата сообщений

- Компонент изолирован в docker-контейнере с Ubuntu 20.04 + ROS2 Foxy Fitzroy
- Общается с хостом на Ubuntu 20.04 + ROS2 Foxy (`docker run -it -d --rm --ipc host image:latest`)



Компонент подписывается на сообщение облака точек (pointcloud2) и публикует массив обнаруженных объектов.

Есть два пакета, которые связаны с обработкой 3D-данных, которые работают только с ROS1, (ROS_numpy и Geometry), и еще используем пользовательские сообщения между топиками.

https://github.com/ros2/common_interfaces/blob/master/sensor_msgs/msg/PointCloud2.msg
https://github.com/eric-wieser/ros_numpy

https://github.com/ros2/geometry2

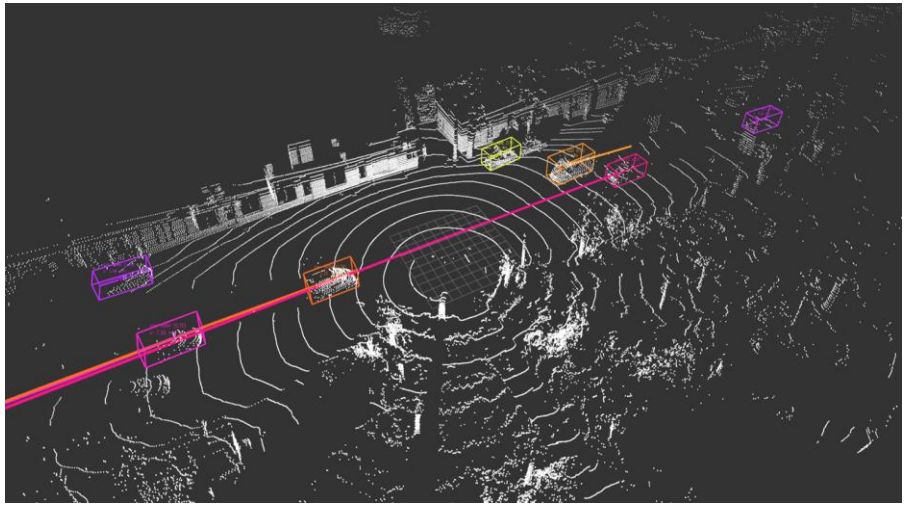
Обнаружение и трекинг объектов в трехмерном облаке точек на основе нейросетевого подхода CenterPoint

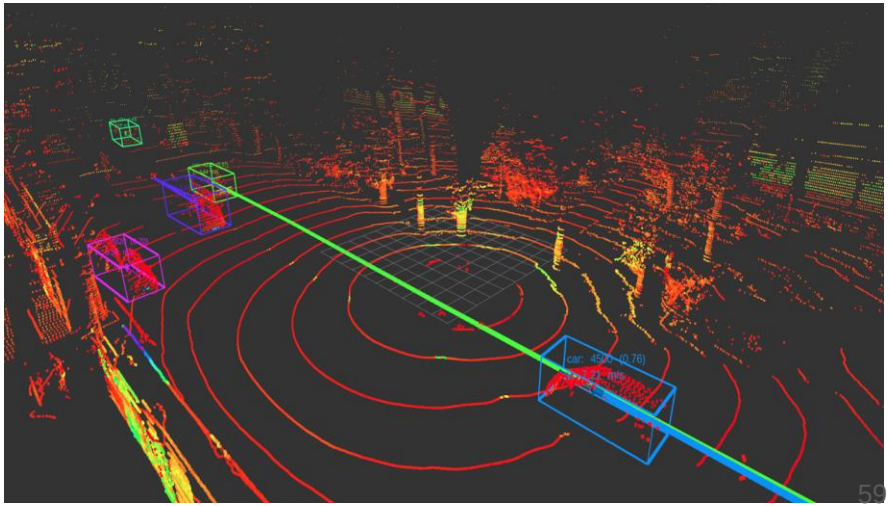
Модифицированная версия библиотеки ros_numpy которая подержит (python3) была использована .

Только нужные функции от пакета TF были добавлены в проект.

поскольку мы используем пользовательские ROS-сообщения, нужно перенести их в ROS2.

Результаты запуска CenterPoint-узла в RVIZ2





Перевод пользовательских ROS-сообщений в ROS2

Сначала создаем новый пакет `ros2 pkg create --build-type ament_cmake objects_msg`

Внутри `objects_msg/msg`, создаем новый файл `object.msg` и добавим нужны письма.

В CMakelists.txt укажем:

```
find_package(rosidl_default_generators REQUIRED)
```

```
rosidl_generate_interfaces(${PROJECT_NAME} "msg/object.msg")
```

поскольку `rosidl_default_generators` нужен для генерации соотвествующего кода, добавим его в `package.xml:` таким образом

```
<build_depend>rosidl_default_generators</build_depend>
```

```
<exec_depend>rosidl_default_runtime</exec_depend>
```

```
<member_of_group>rosidl_interface_packages</member_of_group>
```

собираем пакет : `colcon build --packages-select objects_msg`

https://index.ros.org/doc/ros2/Tutorials/Custom-ROS2-Interfaces

ROS2-Параметры

В предыдущей версии узла, yml-файлы были использованы для работы с параметрами вместе с dynamic configuration (возможность обновления параметров во время выполнения без необходимости перезапуска узла.)

Чтобы их импортировать в ROS2-узел, формат yml-файлов должен быть указан таким образом:

```
/namespace:
  ros_node_name:
    ros__parameters:
      param1: foo
      param2: 10
```

ROSbag2 (.db3)

```
$ ros2 bag record -a(записать bag)

$ ros2 bag record <topic1>(выбирать topic)

$ ros2 bag record -b 10000(разделить по объему)

$ ros2 bag record -d 9000 (разделить по времени)

$ ros2 bag record -a --compression-mode file --
compression-format zstd

$ ros2 bag play <bag_file> -r 0.5(выбирать speed)

$ ros2 bag info <bag_file> (показывать info)
```

https://github.com/ros2/rosbag2

Проигрывание ROS1(melodic) bag файлов в ROS2(dashing)

Работает только для стандартных сообщений

Для поддержки собственных сообщений необходима сборка из исходного кода

ВАЖНО: сначала активация (source) ROS1, затем ROS2

```
sudo apt install \
        ros-dashing-ros2bag \
        ros-dashing-rosbag2 \
        ros-dashing-ros1-bridge \
        ros-dashing-rosbag2-bag-v2-plugins \
        ros-dashing-rosbag2-converter-default-plugins
source /opt/ros/melodic/setup.bash
source /opt/ros/dashing/setup.bash
ros2 bag play -s rosbag_v2 kitti_2011_10_03_drive_0027_synced.bag
```

https://github.com/ros2/rosbag2_bag_v2

ДОПОЛНИТЕЛЬНЫЕ ИСТОЧНИКИ

1. Книга: ROS Robot Programming. YoonSeok Pyo, HanCheol Cho, RyuWoon Jung, TaeHoon Lim (Eng) — основа этой лекции
2. Обучающие инструкции ROS (Eng)
3. Введение в ROS от Voltbro (Rus)
4. Clearpath Robotics ROS Tutorial (Eng)
5. История создания ROS (Eng)

